Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

Лабораторная работа 1

По дисциплине Контейнеризация и оркестрация приложений
Тема работы Знакомство с экосистемой Docker: от базовых команд к диагностике сервиса.
Обучающаяся Мануковская Дарья Михайловна
Факультет Прикладной информаткии
Группа К3321
Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи
Образовательная программа Программирование в инфокоммуникационных системах

Обучающаяся	_____ (дата)	_____ (подпись)	Мануковская Д. М. (Ф.И.О)
Руководитель	_____ (дата)	_____ (подпись)	Аминов Н.С. (Ф.И.О.)

Санкт-Петербург
2026

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ **3**

1 Верификация окружения и архитектуры **4**

2 Первые шаги: «игрушечные» образы **5**

3 Жизненный цикл контейнера **7**

4 Диагностика реального сервиса **8**

ЗАКЛЮЧЕНИЕ **10**

ВВЕДЕНИЕ

Целью практической работы является освоение базовых возможностей Docker: проверка окружения, работа с образами и контейнерами, управление их жизненным циклом и диагностика сервиса. В ходе выполнения заданий применяются стандартные команды Docker CLI с использованием персональных параметров (порт и префикс фамилии). Работа позволяет закрепить навыки запуска, мониторинга и устранения типовых ошибок контейнеров.

1 Верификация окружения и архитектуры

Номер ИСУ: 466625

Порт: 5430 + 25 = 5455

Префикс: manuk

Выполнили 2 команды для просмотра версии и архитектуры. Вывод терминала представлен на рисунке 1.1.

```
docker version --format "Client: {{.Client.Version}}, Server: {{.Server.Version}}"
docker info --format "{{.OSType}}/{{.Architecture}}"
```



Рисунок 1.1 — Терминальный вывод

Далее провели проверку количества образов и контейнеров существующих до работы. (рисунок 1.2)

```
docker system df --format "table {{.Type}}\t{{.TotalCount}}\t{{.Active}}\t{{.Size}}"
```



Рисунок 1.2 — Терминальный вывод

Вопросы:

1. docker version без запуска самого докера выдает ошибку (1.3)
2. docker system prune удаляет все остановленные контейнеры, а также неиспользуемые сети, dangling-образы и dangling build cache.



Рисунок 1.3 — Терминальный вывод

2 Первые шаги: «игрушечные» образы

Запускаем классический пример (рисунок 2.1). Как видно из первых строчек, образ запускается из library/hello-world:latest. Выполняется процесс /hello.

```
docker run --rm hello-world
docker image inspect hello-world --format 'Entrypoint: {{.Config.Entrypoint}}  Cmd: {{.Config.C
```

```
dariamanuk@MacBook-Air-Daria-2 kiop % docker run --rm hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
58dee6a49ef1: Pull complete
c3bdf82c34d1: Download complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (arm64v8)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Рисунок 2.1 — Терминальный вывод

Запускаем минимальный образ. (рисунок 2.2)

```
docker run --rm alpine:3.18 cat /etc/os-release
docker images alpine:3.18 --format "{{.Size}}"
```

```
dariamanuk@MacBook-Air-Daria-2 kiop % docker run --rm alpine:3.18 cat /etc/os-release
Unable to find image 'alpine:3.18' locally
3.18: Pulling from library/alpine
95459497489f: Pull complete
e03f60a96940: Download complete
f6a5f6578c75: Download complete
Digest: sha256:de0eb0b3f2a47ba1eb89389859a9bd88b28e82f5826b6969ad604979713c2d4f
Status: Downloaded newer image for alpine:3.18
NAME="Alpine Linux"
ID=alpine
VERSION_ID=3.18.12
PRETTY_NAME="Alpine Linux v3.18"
HOME_URL="https://alpinelinux.org/"
BUG_REPORT_URL="https://gitlab.alpinelinux.org/alpine/aports/-/issues"
```

Рисунок 2.2 — Терминальный вывод

PRETTY_NAME="Alpine Linux v3.18 11.8MB

Пробуем запустить новый контейнер с удерживанием процесса. (рисунок 2.3)

```
docker run -d --name manuk-alpine-1 alpine:3.18 sleep 300
docker run -d --name manuk-alpine-2 alpine:3.18 sh -c "echo 'Hello from $(hostname)' && sleep 3
docker ps --filter "name=manuk-alpine"
```

```
dariamanuk@MacBook-Air-Daria-2 kiop % docker ps --filter "name=manuk-alpine"
CONTAINER ID   IMAGE          COMMAND                  CREATED              STATUS              PORTS     NAMES
1c78abb3a680   alpine:3.18    "sh -c 'echo 'Hello …"   About a minute ago   Up About a minute             manuk-alpine-2
217bf6650965   alpine:3.18    "sleep 300"              About a minute ago   Up About a minute             manuk-alpine-1
```

Рисунок 2.3 — Терминальный вывод

Вопросы:

1. Запускается программа, и контейнер завершает работу сразу после ее выполнения, sleep 300 ждет конца таймера или до завершения процесса.
2. –rm сразу удаляет контейнер после завершения работы, без него конн-тейнер будет в состоянии exited. Для удаления можно использовать раз-ные команды docker rm name/id или docker container prune.

3 Жизненный цикл контейнера

Проследили статусы контейнеров на каждом этапе его жизненного цикла (рисунок 3.1)

```
docker create --name manuk-nginx nginx:alpine

docker start manuk-nginx

docker stop manuk-nginx

docker restart manuk-nginx

docker kill manuk-nginx

docker rm manuk-nginx

docker ps -a --filter "name=manuk-nginx" --format "table {{.Names}}\t{{.Status}}\t{{.CreatedAt}}
```

```
NAMES         STATUS    CREATED AT
manuk-nginx   Created   2026-10-13 23:38:55 +0300 MSK
manuk-nginx
NAMES         STATUS                  CREATED AT
manuk-nginx   Up Less than a second   2026-10-13 23:38:55 +0300 MSK
manuk-nginx
NAMES         STATUS                                CREATED AT
manuk-nginx   Exited (0) Less than a second ago   2026-10-13 23:38:55 +0300 MSK
manuk-nginx
NAMES         STATUS                  CREATED AT
manuk-nginx   Up Less than a second   2026-10-13 23:38:55 +0300 MSK
manuk-nginx
NAMES         STATUS                                  CREATED AT
manuk-nginx   Exited (137) Less than a second ago   2026-10-13 23:38:55 +0300 MSK
manuk-nginx
NAMES     STATUS    CREATED AT
dariamanuk@MacBook-Air-Daria-2 kiop %
```

Рисунок 3.1 — Терминальный вывод

SIGTERM - graceful shutdown, процесс дается время на корректное завершение и очистку занятых ресурсов, в отличие от SIGKILL, и если процесс записи данных не завершился, то будут удалены при kill.

Вопросы:

1. Ошибка вызова команды представлена на рисунке 3.2
2. Нужен флаг -f -force или сначала остановить его.

```
dariamanuk@MacBook-Air-Daria-2 kiop % docker rm manuk-nginx
Error response from daemon: cannot remove container "manuk-nginx": container is running: stop the container before removing or force remove
```

Рисунок 3.2 — Терминальный вывод

4 Диагностика реального сервиса

После установки образа не выдается ошибка из-за -d, но в диагностике видно. (рисунок 4.1)

```
docker run -d \
  --name manuk-pg-broken \
  -p 5455:5432 \
  postgres:15

docker ps -a --filter "name=manuk-pg"

docker logs --timestamps manuk-pg-broken 2>&1 | tail -20

docker inspect manuk-pg-broken --format='{{.State.ExitCode}}'

```

```
CONTAINER ID   IMAGE         COMMAND                  CREATED         STATUS                     PORTS     NAMES
3b397cfb8320   postgres:15   "docker-entrypoint.s…"   3 minutes ago   Exited (1) 3 minutes ago             manuk-pg-broken
2026-10-13T20:51:09.056471676Z Error: Database is uninitialized and superuser password is not specified.
2026-10-13T20:51:09.056535842Z        You must specify POSTGRES_PASSWORD to a non-empty value for the
2026-10-13T20:51:09.056539551Z        superuser. For example, "-e POSTGRES_PASSWORD=password" on "docker run".
2026-10-13T20:51:09.056540759Z
2026-10-13T20:51:09.056541884Z        You may also use "POSTGRES_HOST_AUTH_METHOD=trust" to allow all
2026-10-13T20:51:09.056543051Z        connections without a password. This is *not* recommended.
2026-10-13T20:51:09.056544009Z
2026-10-13T20:51:09.056544967Z        See PostgreSQL documentation about "trust":
2026-10-13T20:51:09.056546467Z        https://www.postgresql.org/docs/current/auth-trust.html
1
```

Рисунок 4.1 — Диагностика

Исправляем ошибку.

```
docker run -d \
  --name manuk-pg-fixed \
  -e POSTGRES_PASSWORD=Pass_5455 \
  -e POSTGRES_USER=user_466625 \
  -p 5455:5432 \
  postgres:15

sleep 15

docker exec manuk-pg-fixed psql -U user_466625 -c "SELECT version();"
```

Финальный артефакт представлен на рисунке 4.2.

```
echo "=== Практика №1: $(date '+%Y-%m-%d %H:%M:%S') ===" && \
echo "Студент: $(whoami)@$(hostname)" && \
docker ps --filter "name=<фамилия>-pg-fixed" --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}
```

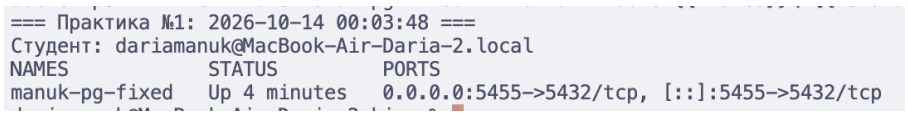


Рисунок 4.2 — Финальный артефакт

ЗАКЛЮЧЕНИЕ

В ходе практической работы выполнены все этапы: проведена верификация окружения, исследованы образы hello-world и Alpine, прослежен жизненный цикл контейнера и выполнена диагностика запуска PostgreSQL. Получены навыки работы с командами управления контейнерами, чтения логов и анализа состояний. Результаты подтверждают понимание основных принципов функционирования Docker и корректного использования официальных образов.